

How the Number-Theoretic Transform Works

A companion to the CSE 220 DFT/FFT offline, Task A bonus

1 The problem the NTT solves

Your FFT-based `multiply` works by treating each big integer as a polynomial (its digits, or packed limbs, are the coefficients), convolving the two coefficient arrays via the convolution theorem, and rounding the result back to integers. That rounding step is the weak point: your radix-2 FFT computes with `complex128` floating point, and a double only carries 53 bits of mantissa. Once a true convolution coefficient exceeds $2^{53} \approx 9 \times 10^{15}$, the transform can no longer represent it exactly, and rounding silently hands you the wrong digit.

The Number-Theoretic Transform (NTT) sidesteps this entirely. Instead of multiplying by complex roots of unity $e^{-2\pi i k/N}$, it multiplies by roots of unity *modulo a prime* p . Every value involved is a plain integer in $\{0, 1, \dots, p-1\}$, every operation is addition, subtraction, or multiplication mod p , and Python integers are exact – so as long as the true (unreduced) coefficients never exceed p , the transform recovers them with zero rounding error, by construction.

2 What a “root of unity mod p ” means

Over the complex numbers, $\omega_N = e^{-2\pi i/N}$ is special because $\omega_N^N = 1$ and its powers $\omega_N^0, \omega_N^1, \dots, \omega_N^{N-1}$ are all distinct – that’s exactly the structure the DFT’s sum needs.

Over the integers mod p , we want the same two properties for some integer g_N : $g_N^N \equiv 1 \pmod{p}$, and its first N powers are all distinct mod p . Such an element exists whenever N divides $p-1$ – a fact from the structure of the multiplicative group mod a prime, which always has a *primitive root* g (an element whose powers $g^0, \dots, g^{p-2}$ enumerate every nonzero residue mod p). Given a primitive root g , an N -th root of unity mod p is

$$g_N = g^{(p-1)/N} \pmod{p}.$$

This is well-defined exactly when $N \mid (p-1)$, which is why the modulus is chosen with a highly composite $p-1$.

The assignment’s suggested modulus is

$$p = 998244353 = 119 \cdot 2^{23} + 1, \quad g = 3 \text{ (a primitive root of } p\text{)}.$$

Because $p-1 = 119 \cdot 2^{23}$, this modulus supports an N -th root of unity for any power-of-two N up to 2^{23} – comfortably larger than any transform length this assignment needs.

3 The forward and inverse transform

The NTT of a sequence $a_0, \dots, a_{N-1}$ (integers mod p) is defined by direct analogy with the DFT:

$$A_k = \sum_{n=0}^{N-1} a_n g_N^{kn} \bmod p, \quad k = 0, \dots, N-1,$$

and the inverse is

$$a_n = N^{-1} \sum_{k=0}^{N-1} A_k g_N^{-kn} \bmod p,$$

where N^{-1} is the modular inverse of $N \bmod p$, and g_N^{-1} is the modular inverse of $g_N \bmod p$ – both computed the same way (Section 5).

Because these sums have exactly the same shape as the DFT sums, the same radix-2 decimation-in-time trick applies unchanged: split into even/odd indexed subsequences, recurse (or iterate with a bit-reversal permutation), and combine with butterflies – just as your `FFTTransformer` does. Concretely, your existing butterfly

$$(g, h) \mapsto (g + h \bmod p, g - h \bmod p)$$

is unchanged in shape; only the twiddle factor changes from a complex exponential ω_M^k to a modular power $g_M^k \bmod p$, and every $+/-/\times$ is followed by `% p`.

4 Why the modulus limits your limb size

For the NTT to give exact answers, the *true* convolution coefficients (before any mod- p reduction) must be $< p$ – otherwise reduction mod p loses information, and there’s no way to recover the true value afterward.

If each limb is base B (i.e. in $\{0, \dots, B-1\}$) and you convolve two length- n limb arrays, the largest possible coefficient is bounded by

$$p_{\max} \leq n(B-1)^2.$$

This is exactly the same bound the assignment already uses to justify $B = 10^4$ for the floating-point path (there, the ceiling is the mantissa, 2^{53}). For the NTT, the ceiling is the modulus $p \approx 9.98 \times 10^8$ instead of $2^{53} \approx 9 \times 10^{15}$ – about seven orders of magnitude smaller – so the same base $B = 10^4$ would badly overflow it. You need to shrink B until $n(B-1)^2 < p$ holds for your largest input (about $n \approx 20000$ limbs); $B = 100$ (two decimal digits per limb) keeps $n(B-1)^2 \approx 2 \times 10^8$, safely under p .

5 Modular inverses via Fermat’s little theorem

The inverse transform needs $N^{-1} \bmod p$ and $g_N^{-1} \bmod p$. Since p is prime, Fermat’s little theorem gives $x^{p-1} \equiv 1 \pmod{p}$ for any $x \neq 0$, which rearranges to

$$x^{-1} \equiv x^{p-2} \pmod{p}.$$

So both inverses are a single modular exponentiation, computed with Python’s built-in three-argument `pow(x, p - 2, p)` – itself an $O(\log p)$ fast-exponentiation routine, exact because everything stays integer.

6 Putting it together: the multiply pipeline

The overall pipeline is identical in shape to the floating-point one – only the transform and the base change:

$$\begin{aligned} a, c &\longrightarrow \text{pad to power of two} \longrightarrow \text{NTT} \longrightarrow A, C \\ &\longrightarrow A_k C_k \bmod p \longrightarrow \text{inverse NTT} \longrightarrow \text{carry sweep} \longrightarrow \text{product}. \end{aligned}$$

Two differences from the floating-point pipeline are worth calling out:

- There is no “take the real part and round” step. The inverse NTT already returns exact integers in $\{0, \dots, p-1\}$, and since you arranged for the true coefficients to be $< p$, those *are* the true coefficients – no reduction of information ever occurred.
- The pointwise multiply $A_k C_k$ must itself be taken mod p at each step, since $A_k, C_k < p$ but their product can be up to $\approx p^2 \approx 10^{18}$ – still exact under Python’s arbitrary-precision integers, just reduced back into $\{0, \dots, p-1\}$ before the inverse transform.

7 Why this is a stronger correctness guarantee

The floating-point FFT can only ever be checked empirically: you compute a result, round it, and compare against ground truth, hoping the rounding error stayed under the threshold for that particular input. The NTT path has no rounding step to hope about – every intermediate value is an exact integer mod p , so if your bound $n(B-1)^2 < p$ holds, the result is guaranteed exact, not just probably close. That’s why the assignment asks you to use it as a cross-check on `inputs/4.txt`: agreement between the two engines is strong evidence that your floating-point FFT’s rounding assumptions actually hold at that size, rather than merely happening to round correctly on the inputs you tried.